Phineas: Robot Móvil Recolector de Cubos para un Sistema Multirrobot Colaborativo sobre ROS 2

Documento técnico de diseño

[Nombre Apellido], [Nombre Apellido], [Nombre Apellido] y [Nombre Apellido]

Equipo Phineas

Escuela de Ingeniería Informática, Universidad Católica Andrés Bello

Robótica

Prof. [Nombre del profesor]

5 de octubre de 2026

Índice

Introducción 4
Planteamiento del Problema 4
Contexto 4
Problema Específico de Phineas 5
Objetivos 5
Restricciones y Supuestos 6
Descripción General del Sistema 6
Stack Tecnológico Propuesto 8
Arquitectura de Hardware 10
Componentes Disponibles y Faltantes 10
Diagrama de Bloques 11
Restricciones Eléctricas Verificadas 12
Mecanismo de Recolección 12
Asignación de Pines y Conexiones 15
Control de los Motores 16
Conexión de Sensores y Actuadores 17
Arquitectura de Software 18
Nodos de ROS 2 18
Percepción de Cubos por Color 19
Cinemática Mecanum 20
Máquina de Estados 20
Convivencia en el Espacio Compartido 22
Detección del Otro Robot 23
Esquive sin Detenerse 24
Ampliación con Cámara Cenital 25
Comunicación con Ferb y Perry 26
Red y Descubrimiento 26
Contrato de Interfaces de Phineas 27
Protocolo de Acceso a la Zona de Transferencia 27
Simulación en Webots 29
Del Simulador al Robot Real 30
Riesgos y Verificaciones Pendientes 31
Plan de Pruebas 32

Conclusiones 33
Referencias 34

Phineas: Robot Móvil Recolector de Cubos para un Sistema Multirrobot Colaborativo sobre ROS 2

Introducción

El proyecto de Robótica de este período consiste en un sistema de tres robots que cooperan sin una computadora central. Dos robots móviles, Phineas y Ferb, recogen cubos de colores en una pista que comparten, y un brazo robótico fijo, Perry, los toma de una zona de transferencia y los clasifica. Cada robot tiene su propia Raspberry Pi 5 y todos se comunican sobre ROS 2 (Robot Operating System 2, el middleware de robótica que gestiona procesos, mensajes y descubrimiento en red).

Este documento presenta el diseño de **Phineas**: el problema que resuelve, el stack tecnológico, el hardware disponible y el faltante, la asignación de pines, el mecanismo de captura, la arquitectura de software, la convivencia con Ferb en el espacio compartido, la comunicación con los otros robots y la simulación en Webots. Cada decisión se acompaña de su motivo y de la alternativa descartada, y los datos de hardware provienen de hojas de datos o del modelo de simulación del grupo.

Planteamiento del Problema

Contexto

La pista es una superficie blanca delimitada por una línea negra, con un cuadro negro que funciona como zona de transferencia entre los robots móviles y el brazo. Sobre ella hay cubos de seis colores. **Phineas y Ferb comparten todo el espacio**: no hay zonas asignadas, por lo que pueden encontrarse en cualquier punto. El trabajo se reparte por color: Phineas recoge los cubos rojos, naranjas y amarillos, y Ferb los verdes, azules y morados. Perry recibe cada cubo en el

cuadro y lo deposita en el contenedor que corresponde. La evaluación es colectiva y debe registrarse cuántos cubos entregó cada robot (Ignacio, 2026a).

Problema Específico de Phineas

Phineas debe localizar los cubos de sus tres colores, capturarlos sin perderlos durante el trayecto, llevarlos al cuadro y soltarlos dentro, todo en un espacio donde Ferb se mueve al mismo tiempo y donde solo un robot puede usar el cuadro a la vez. Su chasis mecanum usa motores TT de reducción 1:120 sin encoders (LewanSoul, s. f.), es decir, sin medición directa de cuánto avanzó. Eso obliga a una navegación **reactiva**, guiada por sensores y no por odometría (estimación de la posición integrando el giro de las ruedas).

Objetivos

Objetivo General

Diseñar un robot móvil autónomo sobre Raspberry Pi 5 y ROS 2 que recolecte los cubos de sus colores y los entregue en la zona de transferencia, conviviendo con Ferb en la misma pista y coordinado con Perry sin intervención humana.

Objetivos Específicos

1. Definir el hardware faltante y su conexión eléctrica segura con la Raspberry Pi 5 y el HAT Cokoino.
2. Implementar la detección y clasificación por color de los cubos y la detección del borde de la pista.
3. Implementar una máquina de estados que encadene búsqueda, captura, verificación de color, transporte y entrega.
4. Evitar colisiones con Ferb mediante reglas de circulación, exclusión mutua del cuadro y frenado reactivo.

5. Validar el comportamiento en Webots antes de probarlo en el robot real, con la misma interfaz de ROS 2.

Restricciones y Supuestos

Tabla 1

Restricciones y supuestos de diseño

Restricción o supuesto	Origen	Consecuencia en el diseño
Evitar cortocircuitos; conectar todo con los equipos apagados; operar bajo supervisión de Wilfredo	Reglas del laboratorio	Fusible e interruptor maestro en la línea de batería; montaje verificado antes de energizar
Raspberry Pi 5 y ROS 2 en cada robot; las Pi se comunican entre sí sin PC central	Enunciado del proyecto	Coordinación distribuida; el árbitro del cuadro vive en la Pi de Perry
Phineas y Ferb comparten toda la pista	Enunciado del proyecto	Reglas de circulación, frenado reactivo y prioridad fija entre robots
Reparto por colores: Phineas rojo, naranja y amarillo	Decisión del grupo; por aprobar con el profesor	Clasificación de color con cámara y confirmación con el TCS34725
Cubos de unos 2,5 cm (estimado en Webots, por medir)	Modelo de simulación del grupo	Pala en U de 32 mm; si los cubos reales son mayores, se rediseña
Motores TT 1:120 de 3–6 V, sin encoders	Ficha del kit (LewanSoul, s. f.)	Navegación reactiva; límite del ciclo de trabajo del PWM
HAT documentado para Pi 4B y anteriores, no para Pi 5	Manual de Cokoino (s. f.-b)	Portar el control a lgpio y verificarlo en banco

Descripción General del Sistema

La Figura 1 muestra el flujo de un cubo. Phineas y Ferb son idénticos en hardware y software; los diferencia solo su archivo de configuración (nombre, colores asignados y prioridad). El cuadro es el único recurso que deben compartir por turnos, y la pista es un espacio común donde pueden cruzarse.

Figura 1

Flujo de un cubo a través de los tres robots

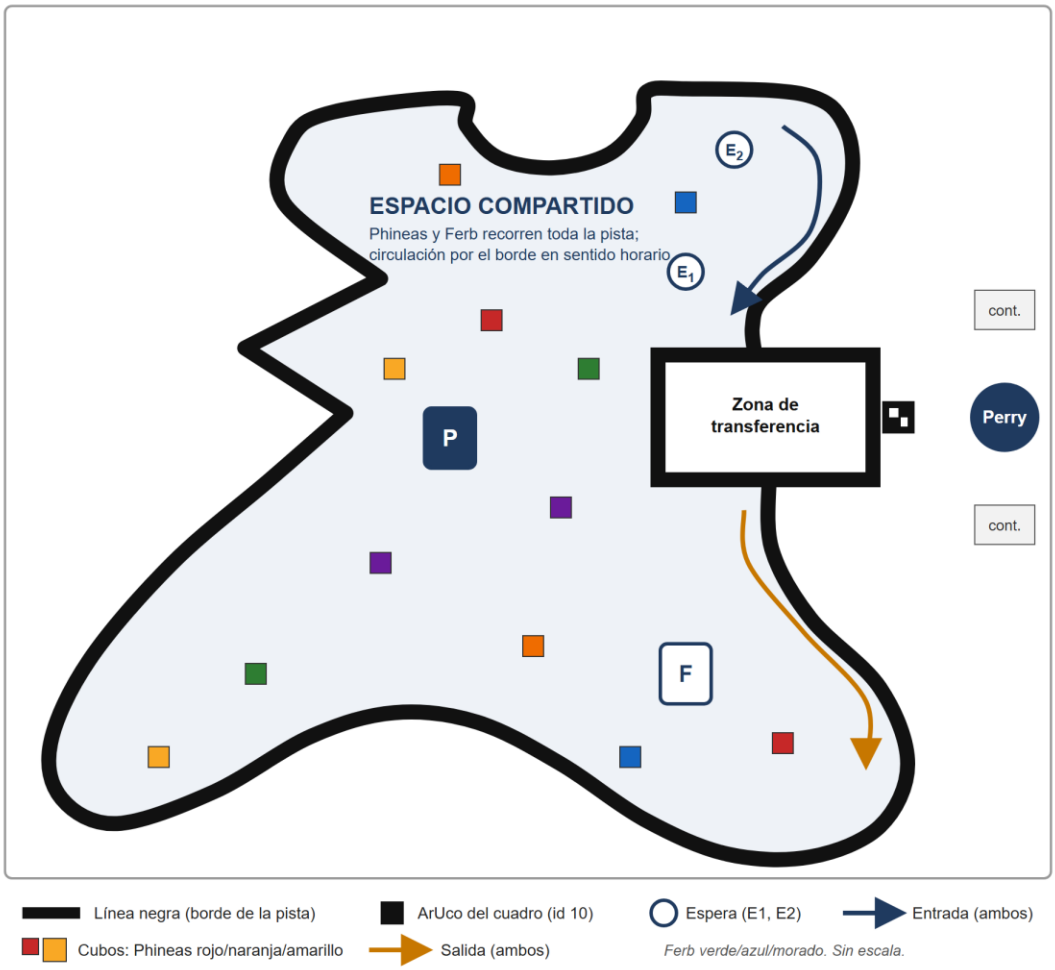


Nota. Elaboración propia.

La Figura 2 muestra la pista compartida. Los dos robots buscan en toda la superficie. Ninguno espera al otro para moverse: cuando se encuentran, se esquivan con las reglas de la sección de convivencia. Solo hay dos reglas de tránsito: quien sigue el borde para orientarse lo hace en sentido horario, y el cuadro se usa por turnos, porque mide unos 24 cm y cada robot ocupa unos 19 × 16 cm, así que no caben dos ni queda espacio para el brazo. El robot que llega mientras el otro entrega espera unos segundos en E_1; un segundo, en E_2. Un marcador ArUco (código visual cuadrado que la cámara identifica y del que puede estimar la posición) junto al cuadro ayuda a la aproximación final; su uso depende de que las reglas lo permitan.

Figura 2

Espacio compartido, sentido de circulación y puntos de espera



Nota. Trazado aproximado a partir de la fotografía de la pista; sin escala.

Stack Tecnológico Propuesto

La Tabla 2 resume el stack por capa. El criterio común es minimizar el tiempo de depuración de un equipo que aprende ROS 2 mientras construye el robot, por eso se prefieren las opciones con instalación oficial y documentación abundante.

Tabla 2

Stack tecnológico de Phineas

Capa	Elección	Motivo	Alternativa descartada
Sistema operativo	Ubuntu Server 24.04 LTS (arm64)	Plataforma Tier 1 de ROS 2 Jazzy: se instala por paquetes oficiales (Open Robotics, 2024)	Raspberry Pi OS: sin binarios oficiales de ROS 2; exige Docker o compilar
Middleware	ROS 2 Jazzy Jalisco (LTS hasta mayo de 2029)	Tópicos, servicios y acciones entre las tres Pi; herramientas de depuración incluidas	Protocolo propio sobre MQTT: habría que construir descubrimiento, QoS y herramientas
Transporte (RMW)	Fast DDS por defecto; `rmw_zenoh_cpp` como respaldo	Zenoh no depende de multicast (ros2, s. f.)	—
Lenguaje	Python 3.12 con `rclpy`	Decisiones a pocos hercios; OpenCV y NumPy ya ejecutan en C/C++	C++: solo para un nodo que no cumpla su latencia
GPIO	`lgpio` (con `gpiozero`)	Única fábrica de pines de gpiozero compatible con Pi 5 (GPIO Zero, s. f.)	`RPi.GPIO`, usado por el ejemplo de Cokoino: no funciona en Pi 5
PWM del servo	PWM hardware del RP1 en GPIO18	El PWM por software produce fluctuación que calienta el servo (joan2937, s. f.)	`tx_servo` de lgpio
Cámara	IMX219 de 120° por CSI (cable de 22 pines incluido)	Gran angular: ve la pala y el entorno a la vez (Waveshare, s. f.-a)	Webcam USB: ocupa puerto y presupuesto de corriente USB
Visión	OpenCV: HSV, contornos y ArUco	Clasificación por color sin entrenamiento; ArUco para ubicar el cuadro y al otro robot	YOLO: requiere un conjunto de datos etiquetado
Simulación	Webots R2025a + `webots_ros2`	El mundo de la pista y los dos robots ya existen; mismos tópicos que el robot real	Gazebo: habría que modelar todo desde cero
Código y sincronía	Git + GitHub, un solo repositorio; chrony	Un cambio de contrato llega a los tres robots en un commit; marcas de tiempo coherentes	Un repositorio por subsistema

Quedan fuera a propósito Nav2 y SLAM, porque sin encoders no hay odometría fiable que los alimente, y las redes neuronales de detección, porque requieren datos etiquetados que hoy no existen. Una red YOLO puede añadirse después sin cambiar el resto del sistema si publica el mismo mensaje (Ignacio, 2026b).

Arquitectura de Hardware

Componentes Disponibles y Faltantes

El chasis, la Raspberry Pi 5 con su disipador activo, el HAT de motores y la cámara ya están en el laboratorio. La Tabla 3 lista el material mínimo con su estado. Los sensores coinciden con los del modelo de Webots, de modo que lo que se pruebe en simulación corresponde a piezas reales.

Tabla 3

Componentes de Phineas

Componente	Función	Estado	Prioridad
Raspberry Pi 5, 8 GB, con Active Cooler	Cómputo y nodos ROS 2; evitar reducción de frecuencia por calor	Disponible	—
HAT Cokoino 4WD (2 × DRV8833 + regulador GS2678)	Puentes H para 4 motores; 5,1 V para la Pi	Disponible, sin verificar en Pi 5	—
Chasis mecanum LewanSoul/Hiwonder, 4 motores TT 1:120	Movimiento omnidireccional	Disponible	—
Cámara Pi5-IMX219-120 y carcasa Arducam	Detectar cubos, su color, el cuadro y su ArUco	Disponible	—
Portapilas 2 × 18650, celdas y cargador	Alimentación (7,4 V nominal)	Por confirmar	Imprescindible
4 módulos TCRT5000	Detectar la línea negra	Faltante	Imprescindible
HC-SR04 frontal	Detectar a Ferb y obstáculos	Faltante	Imprescindible
4 marcadores ArUco laterales impresos	Que la cámara de Ferb identifique a Phineas desde cualquier lado	Faltante	Imprescindible
Servo SG90 y pala en U impresa en PLA	Capturar, retener y soltar el cubo	Faltante	Imprescindible
Sensor IR FC-51	Detectar que hay un cubo en la pala	Faltante	Imprescindible
Sensor de color TCS34725	Confirmar el color del cubo capturado	Faltante	Imprescindible
IMU MPU-6050	Mantener el rumbo; los mecanum derrapan	Faltante	Imprescindible
Interruptor maestro, portafusible y fusible de 5 A	Corte de energía y protección	Faltante	Imprescindible
microSD A2 de 32 GB; resistencias y condensador de 470 µF	Sistema operativo; divisores de tensión; filtro del servo	Faltante	Imprescindible

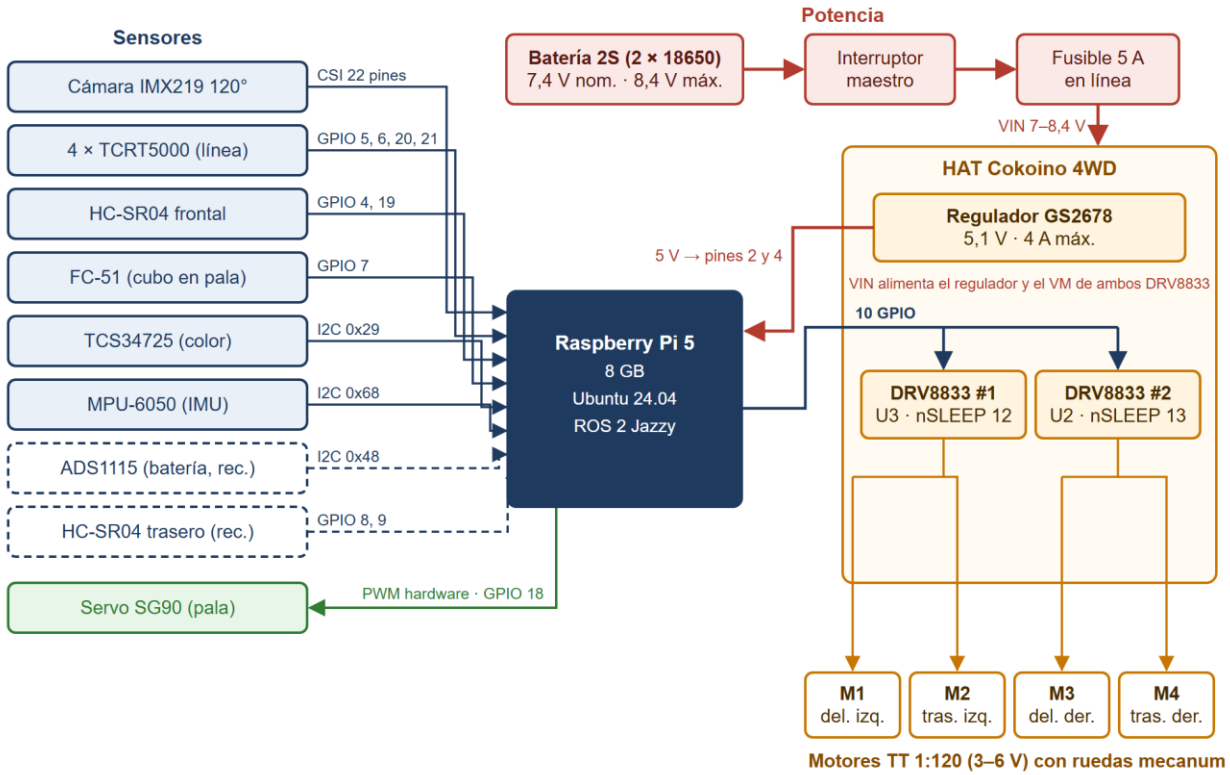
Componente	Función	Estado	Prioridad
HC-SR04 trasero	Ver a Ferb al retroceder; la cámara no ve hacia atrás	Faltante	Recomendado
Convertidor ADC ADS1115	Medir el voltaje de la batería (la Pi no tiene ADC)	Faltante	Recomendado

Diagrama de Bloques

La Figura 3 separa la ruta de potencia (rojo), que lleva la batería a los puentes H y al regulador, de la ruta de señal (azul). El regulador GS2678 del HAT entrega 5,1 V y hasta 4 A a la Pi por los pines 2 y 4 del cabezal, y los dos DRV8833 toman VM directamente de la batería (Cokoino, s. f.-b). La cámara usa el conector CSI propio de la Pi y no ocupa pines del cabezal.

Figura 3

Diagrama de bloques de hardware de Phineas



Nota. Los bloques con borde discontinuo son recomendados. Elaboración propia a partir del esquemático de Cokoino (s. f.-a).

Restricciones Eléctricas Verificadas

- **Solo batería 2S.** El DRV8833 opera entre 2,7 V y 10,8 V, con máximo absoluto de 11,8 V (Texas Instruments, 2015). Una batería 3S (12,6 V cargada) destruye los drivers aunque el HAT acepte 12 V.
- **Ciclo de trabajo limitado.** Los motores TT están especificados para 3–6 V y reciben el voltaje de la batería. El PWM no debe superar `duty_max = 6 V / V_bat` (71 % con la batería llena), el mismo límite que usa el modelo de Webots.
- **Presupuesto de 5 V.** El regulador del HAT da 4 A, por debajo de los 5 A recomendados para la Pi 5; sin negociación USB-PD la Pi limita los periféricos USB a 600 mA (Raspberry Pi Ltd, s. f.). La cámara CSI no consume de ese presupuesto USB; no debe activarse `usb_max_current_enable`.
- **Corriente por puente.** El DRV8833 del HAT es el encapsulado TSSOP, que admite 500 mA RMS y 2 A de pico por puente, con protección por sobrecorriente entre 2 A y 3,3 A (Texas Instruments, 2015). Un motor bloqueado contra Ferb o el borde puede dispararla; el software detecta el atasco y corta tras un tiempo máximo.
- **Rampas de aceleración.** El arranque de cuatro motores hunde la tensión de la batería y puede reiniciar la Pi y corromper la microSD. El controlador limita la variación de velocidad por ciclo, y se mide el pin 2 con multímetro durante un arranque brusco.

Mecanismo de Recolección

El chasis por sí solo no retiene un cubo: en cualquier giro o desplazamiento lateral, frecuentes en un mecanum, el cubo se escapa. La Tabla 4 compara las opciones.

Tabla 4

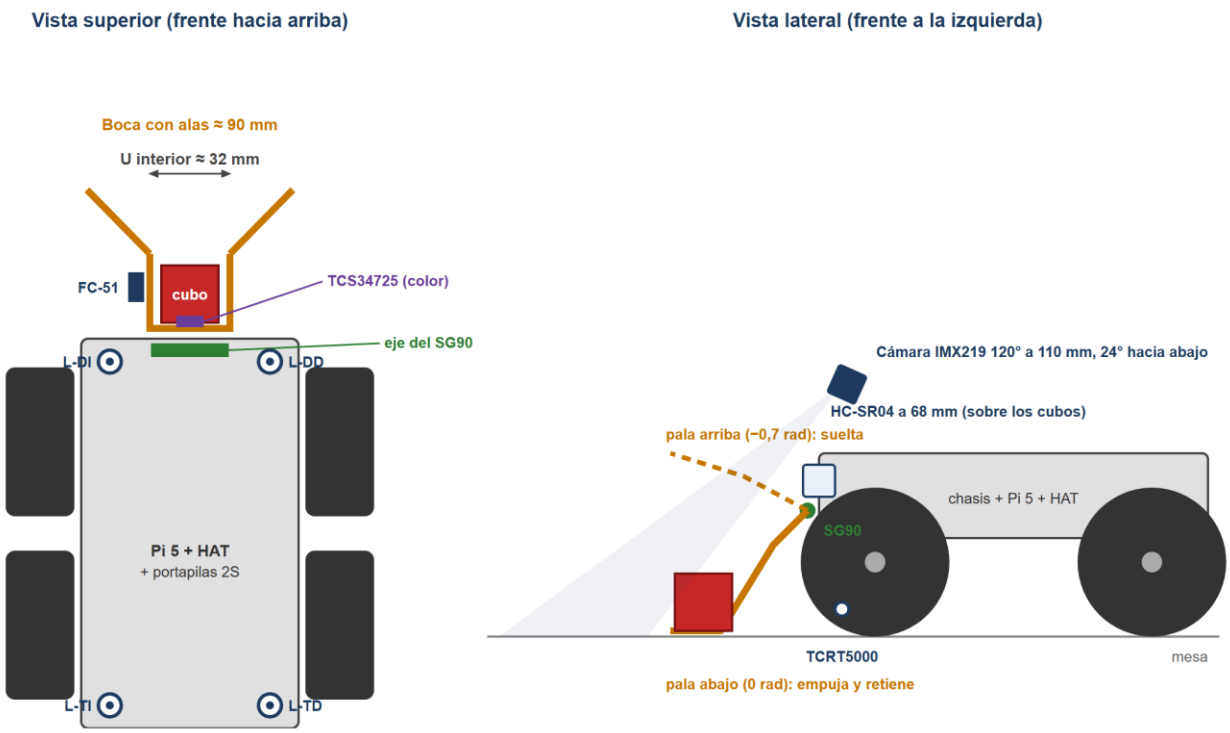
Comparación de mecanismos de captura

Criterio	Empuje directo	Pala en U con elevación	Pinza servoaccionada
Retención en giros y en reversa	Nula	Alta: la U rodea el cubo	Total
Precisión de alineación requerida	Alta	Baja: las alas guían el cubo a la U	Alta
Tolerancia a tamaños distintos	Alta	Limitada a cubos menores de 32 mm	Media
Actuadores	0	1 servo	1–2 servos
Entrega en el cuadro	Retroceder	Subir la pala y retroceder	Abrir y levantar
Estado en el proyecto	—	Modelada y probada en Webots	—

Se adopta la **pala en U con elevación** del modelo de Webots (Figura 4). Es una captura pasiva: las alas de unos 90 mm encauzan el cubo hacia una U interior de unos 32 mm, y el servo SG90 solo tiene dos posiciones: abajo para empujar y retener, y arriba (unos −0,7 rad) para soltar. Un error de percepción de algunos centímetros no provoca un fallo de agarre, porque la geometría corrige la alineación. La pinza sería necesaria si hubiera que levantar el cubo, pero eso lo hace Perry.

Figura 4

Pala en U con elevación y ubicación de sensores



Nota. Medidas tomadas del modelo RecolectorUCAB de Webots; se ajustan al medir los cubos reales.

Dentro de la pala trabajan dos sensores con funciones distintas:

- El **FC-51** es un sensor infrarrojo reflectivo con salida digital, activa en bajo, y alcance ajustable de 2 a 30 cm (Cirkit Designer, s. f.). Indica que hay un cubo en la U. Su lectura depende de cuánto infrarrojo refleja el objeto: los colores de Phineas son claros y reflejan bien, pero conviene probarlo con todos los colores, porque un cubo morado oscuro podría no detectarse y ese caso afecta sobre todo a Ferb.
- El **TCS34725** es un sensor de color I2C (dirección 0x29) que se alimenta a 3,3 V (Waveshare, s. f.-b). Confirma de cerca el color que la cámara estimó de lejos. Si el cubo no es de Phineas, el robot sube la pala, retrocede y marca ese cubo como ignorado.

- El **HC-SR04** va a 68 mm de altura, por encima de los cubos, para que detecte a Ferb y a Perry, pero no los cubos que Phineas quiere capturar.

La pala del modelo es naranja. Si se imprime de ese color, la cámara de Phineas puede confundir la pala de Ferb con un cubo naranja; por eso se recomienda imprimirla en gris o negro.

Asignación de Pines y Conexiones

El HAT ocupa diez pines fijos: GPIO12 y GPIO13 para nSLEEP, y GPIO16, 17, 22, 23, 24, 25, 26 y 27 para las entradas de los puentes H (Cokoino, s. f.-b). El sensor de color, la IMU y el ADC comparten el bus I2C porque tienen direcciones distintas, lo que ahorra pines. La Tabla 5 y la Figura 5 muestran la asignación completa.

Tabla 5

Asignación de pines de la Raspberry Pi 5 en Phineas

Función	GPIO (BCM)	Pin físico	Nota
DRV8833 #1: nSLEEP / AIN1 / AIN2 / BIN1 / BIN2	12 / 17 / 27 / 22 / 23	32 / 11 / 13 / 15 / 16	M1 y M2 (lado izquierdo); fijado por el HAT
DRV8833 #2: nSLEEP / AIN1 / AIN2 / BIN1 / BIN2	13 / 24 / 25 / 26 / 16	33 / 18 / 22 / 37 / 36	M3 y M4 (lado derecho); fijado por el HAT
Línea: delanteros izq. / der.	5 / 6	29 / 31	TCRT5000 a 3,3 V
Línea: traseros izq. / der.	20 / 21	38 / 40	TCRT5000 a 3,3 V
HC-SR04 frontal: TRIG / ECHO	4 / 19	7 / 35	ECHO con divisor 1 kΩ / 2 kΩ
HC-SR04 trasero (recomendado): TRIG / ECHO	8 / 9	24 / 21	ECHO con divisor 1 kΩ / 2 kΩ
Servo SG90 de la pala	18	12	PWM hardware del RP1
FC-51: cubo en la pala	7	26	Módulo a 3,3 V; activo en bajo
I2C: TCS34725 (0x29), MPU-6050 (0x68), ADS1115 (0x48)	2 / 3	3 / 5	Bus compartido a 3,3 V
Cámara IMX219	—	Conector CSI	No usa el cabezal
Consola serie (reservado)	14 / 15	8 / 10	Depuración sin red

Figura 5

Mapa del cabezal de 40 pines de Phineas



Nota. Pines del HAT según el manual de Cokoino (s. f.-b); canales de PWM hardware del Pi 5 según Gadgetoid (s. f.).

Control de los Motores

El código de ejemplo del fabricante regula la velocidad aplicando PWM de 1 kHz sobre nSLEEP (Cokoino, s. f.-a). La hoja de datos indica que el puente tarda hasta 1 ms en activarse después de que nSLEEP sube (t_{WAKE}), que es exactamente un periodo a 1 kHz, así que la relación entre ciclo de trabajo y velocidad resulta errática. Además, un solo nSLEEP por chip obliga a que dos ruedas giren siempre a la misma velocidad.

La corrección es mantener nSLEEP fijo en alto y aplicar el PWM sobre las entradas xIN1/xIN2, algo que la hoja de datos admite de forma explícita: PWM en xIN1 con xIN2 en bajo produce avance con decaimiento rápido, y a la inversa para retroceder (Texas Instruments, 2015).

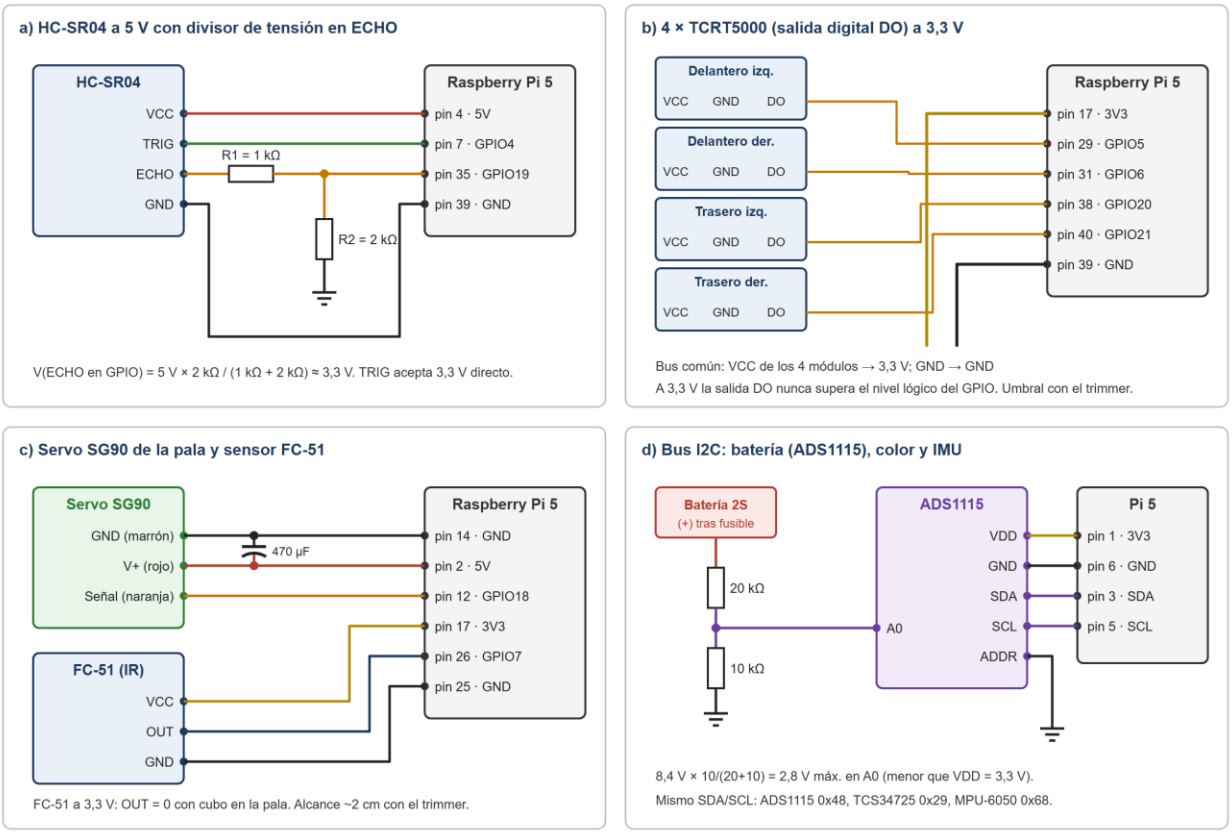
Las entradas aceptan 2 V como nivel alto, por lo que bastan los 3,3 V de la Pi. Así cada rueda tiene velocidad independiente, que es lo que exige la cinemática mecanum. Si el PWM por software de lgpio no alcanzara, el plan de respaldo es PWM de baja frecuencia sobre nSLEEP; como cada lado del robot está en un chip distinto, se conserva el giro diferencial.

Conexión de Sensores y Actuadores

La Figura 6 detalla las conexiones que requieren algún componente adicional. Los pines de la Pi trabajan a 3,3 V (Raspberry Pi Ltd, s. f.). El ECHO del HC-SR04, que opera a 5 V (SparkFun Electronics, s. f.), pasa por un divisor de tensión. Los TCRT5000 y el FC-51 se alimentan a 3,3 V para que su salida nunca supere ese nivel, y la batería se mide con un divisor 1:3 hacia el ADS1115.

Figura 6

Esquemas de conexión de sensores y actuadores



Nota. Todas las conexiones se realizan con la batería desconectada y el interruptor maestro abierto, conforme a las reglas del laboratorio.

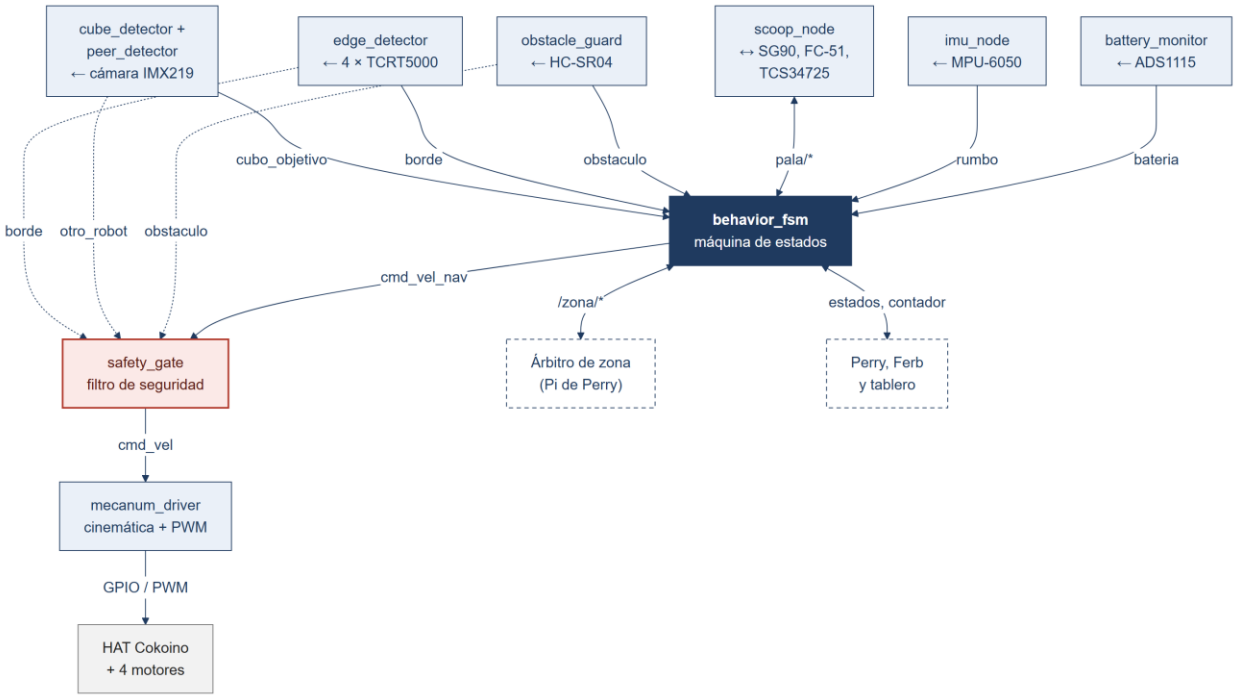
Arquitectura de Software

Nodos de ROS 2

El software de Phineas es un paquete ROS 2 compartido con Ferb (`mobile_base`), con un nodo por responsabilidad (Figura 7). Los sensores publican hechos (hay un borde, hay un cubo rojo a 20 cm, Ferb está a 40 cm a la izquierda), la máquina de estados decide, y un único nodo toca los motores. Esta separación permite que los mismos nodos funcionen en Webots y en el robot real.

Figura 7

Grafo de nodos y tópicos de Phineas



Nota. Los tópicos locales llevan el prefijo /phineas/, que se omite en el diagrama. Elaboración propia.

El nodo `safety_gate` aplica el patrón **safety filter** (filtro de seguridad): toda orden de movimiento pasa por él, y la anula, la reduce o le suma un desplazamiento lateral si un sensor de línea ve el borde, el ultrasonido ve algo cerca o `peer_detector` informa que Ferb se acerca. Así un error en la lógica de alto nivel no puede sacar al robot de la pista ni empujarlo contra Ferb, y el esquive ocurre sin que la máquina de estados cambie de tarea. A su vez, `mecanum_driver` detiene los motores si no recibe órdenes durante 300 ms (**dead-man switch**, o parada por inactividad).

Percepción de Cubos por Color

El nodo `cube_detector` convierte la imagen de la cámara a HSV, separa los píxeles de saturación alta del fondo blanco y busca contornos. Conserva solo los que tienen área de cubo, proporción cercana a 1 y solidez alta; ese filtro descarta la línea negra, que es alargada, y el

contorno del cuadro, que es hueco. A cada contorno le asigna un color según su tono, y solo los rojos, naranjas y amarillos pasan a ser objetivos de Phineas. La distancia se estima con la fila inferior del contorno mediante una tabla calibrada con la cámara montada. El controlador de Webots del grupo ya clasifica por tono con este mismo criterio.

Una lente gran angular como la de 120° deforma la imagen hacia los bordes, por lo que la cámara se calibra una vez con un tablero de ajedrez antes de estimar distancias o la posición del ArUco. Los rangos de color viven en un archivo YAML y se recalibran con la luz real del laboratorio, que el simulador no reproduce.

Cinemática Mecanum

El nodo `mecanum_driver` convierte la velocidad deseada del robot (v_x hacia adelante, v_y hacia la izquierda, ω_z de giro) en la velocidad de cada rueda con el modelo cinemático inverso de Taheri et al. (2015), donde r es el radio de la rueda y l_x, l_y son las semidistancias entre ejes:

$$\omega_{FI} = (v_x - v_y - (l_x + l_y)\,\omega_z) / r \qquad \omega_{FD} = (v_x + v_y + (l_x + l_y)\,\omega_z) / r$$

$$\omega_{TI} = (v_x + v_y - (l_x + l_y)\,\omega_z) / r \qquad \omega_{TD} = (v_x - v_y + (l_x + l_y)\,\omega_z) / r$$

Sin encoders, estas velocidades se aplican en lazo abierto, normalizadas para que ninguna rueda supere `duty_max`. El giroscopio de la MPU-6050 corrige el rumbo durante los desplazamientos rectos; como no tiene brújula, el rumbo integrado deriva y solo es fiable en maniobras cortas. Los signos dependen de la orientación de los rodillos, así que se validan primero con una prueba de desplazamiento lateral, en Webots y luego en el robot.

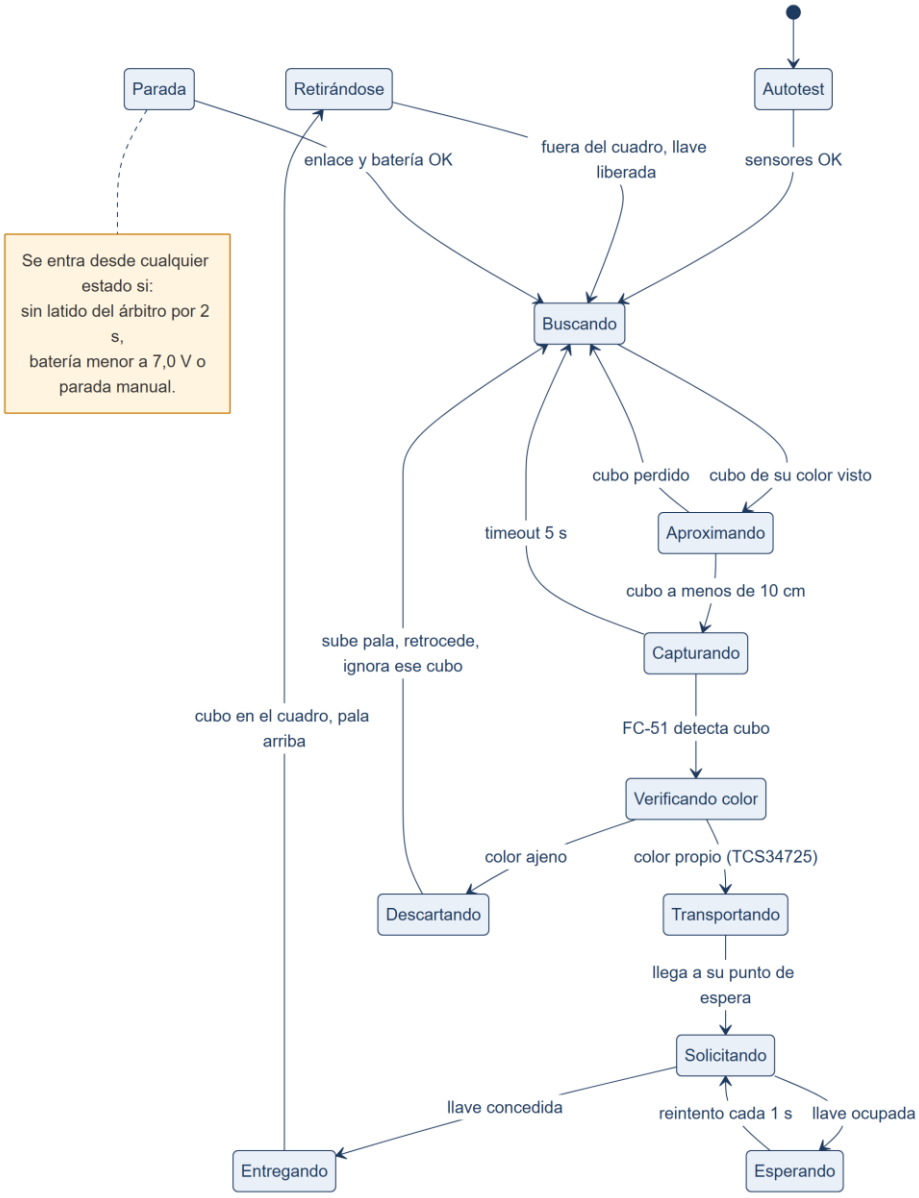
Máquina de Estados

El nodo `behavior_fsm` implementa la **finite state machine** (máquina de estados finita) de la Figura 8. Cada estado tiene un criterio de salida medible y un tiempo máximo, de modo que el robot nunca queda esperando indefinidamente un evento que no llega. El estado **Verificando**

color existe porque la cámara clasifica de lejos y puede equivocarse con la iluminación; el TCS34725 confirma de cerca antes de invertir un viaje al cuadro.

Figura 8

Máquina de estados de Phineas



Nota. Los umbrales (10 cm, 5 s, 7,0 V) son valores iniciales configurables en YAML. Elaboración propia.

En **Buscando**, Phineas gira sobre sí mismo para cubrir su entorno con la cámara y avanza hacia el cubo propio más cercano; si no ve ninguno, recorre la pista bordeándola en sentido

horario. En **Transportando**, busca el cuadro o su ArUco con la cámara y, si no lo ve, sigue el borde en sentido horario hasta encontrarlo, de modo que siempre llega por el lado norte. En **Entregando**, avanza hasta que los sensores delanteros detectan el borde del cuadro, entra lo justo para que el cubo quede dentro, sube la pala y sale por el sur antes de liberar la llave.

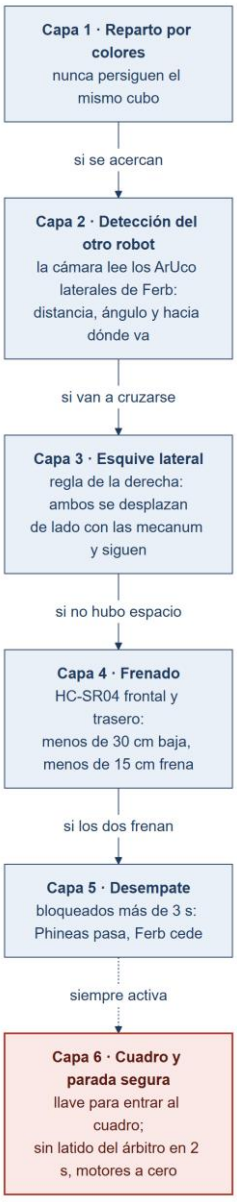
Convivencia en el Espacio Compartido

La consigna es que Phineas y Ferb anden libremente por toda la pista al mismo tiempo. Para esquivarse, cada robot necesita saber dónde está el otro, y la comunicación por sí sola no lo resuelve: un robot solo puede transmitir lo que sabe, y sin encoders ni cámara cenital ninguno conoce su propia posición. Por eso la evasión se basa en **percepción local**: cada robot ve al otro con su propia cámara y sus ultrasonidos, y la red se usa para compartir intenciones y la regla de prioridad, no posiciones.

El diseño apila seis capas (Figura 9). Cada una reduce los casos que llegan a la siguiente, y un choque solo ocurre si fallan todas; el enfoque se conoce como **defense in depth** (defensa en profundidad).

Figura 9

Capas de convivencia entre Phineas y Ferb



Nota. Elaboración propia a partir del esquema de capas del documento inicial del grupo (Ignacio, 2026a).

Detección del Otro Robot

Cada robot lleva cuatro marcadores ArUco impresos, uno por lado, con un identificador distinto por cara (por ejemplo, 11–14 para Phineas y 21–24 para Ferb). El nodo `peer_detector` los busca en la misma imagen que usa `cube_detector`. Con la cámara calibrada, cada marcador

detectado entrega la distancia y el ángulo al otro robot, y la cara visible indica hacia dónde mira: si se ve la cara frontal de Ferb, Ferb viene de frente. Comparando dos o tres imágenes seguidas se estima si se acerca y en qué dirección. Con 120° de campo de visión, la cámara cubre todo el frente y los costados delanteros; lo que queda atrás lo cubre el HC-SR04 trasero recomendado.

Los marcadores que hoy tiene el modelo de Webots están encima del robot, donde la cámara del otro, a 11 cm de altura, no los ve. Hay que agregar al `RecolectorUCAB.proto` los cuatro marcadores laterales, y comprobar en Webots y en la pista a qué distancia máxima se detectan con el tamaño elegido.

Esquive sin Detenerse

Cuando `peer_detector` predice que los robots se van a cruzar a menos de 30 cm, `safety_gate` suma a la orden de la máquina de estados un desplazamiento lateral hacia la derecha del robot. Ambos robots aplican la misma regla, igual que dos personas que se cruzan en un pasillo y se apartan cada una hacia su derecha: la regla es simétrica, ninguno espera, y no hace falta negociar por la red quién se aparta. Las ruedas mecanum permiten ese desplazamiento lateral sin girar, de modo que cada robot conserva el rumbo hacia su cubo y lo retoma en cuanto pasa el otro. Si a la derecha está el borde de la pista, manda el sensor de línea y el robot frena en vez de apartarse.

- **Reparto por colores.** Al no compartir objetivos, los robots nunca compiten por el mismo cubo, que sería el encuentro más frecuente.
- **Detección del otro robot.** La cámara identifica a Ferb por sus ArUco laterales y estima si viene hacia Phineas.
- **Esquive lateral.** Regla de la derecha con desplazamiento mecanum; ninguno se detiene.

- **Frenado.** El ultrasonido frontal reduce la velocidad a menos de 30 cm y frena a menos de 15 cm; el trasero cubre los retrocesos. Actúa aunque la cámara no haya visto a Ferb, por ejemplo con mala luz.
- **Desempate.** Si ambos quedan frenados frente a frente más de 3 s, Phineas pasa y Ferb cede. Sin una regla asimétrica para este caso ambos cederían a la vez y volverían a bloquearse, un **livelock** (bloqueo activo).
- **Cuadro y parada segura.** El cuadro se usa por turnos con la llave del árbitro y, si se pierde el latido del árbitro, los dos robots se detienen en menos de 2 s.

La red complementa esta percepción con intenciones: cada robot publica su estado (buscando, transportando, entregando) en `/phineas/estado` y `/ferb/estado`. Si Phineas sabe que Ferb está transportando hacia el cuadro, puede anticipar por dónde pasará y no elegir un cubo en esa trayectoria.

Ampliación con Cámara Cenital

Si las reglas lo permiten, una sola cámara fija sobre la pista, procesada en la Pi de Perry, podría leer los ArUco superiores de ambos robots (identificadores 1 y 2 en Webots) y publicar sus posiciones. Con posiciones compartidas, los robots podrían aplicar evasión recíproca (cada uno se aparta la mitad, asumiendo que el otro hace lo mismo) antes incluso de verse. No reemplaza a la percepción local: si esa cámara falla, el sistema debe seguir funcionando con las capas anteriores. Es la misma cámara cenital que el documento inicial del grupo propone para Perry, colocada lo bastante alta para cubrir toda la pista.

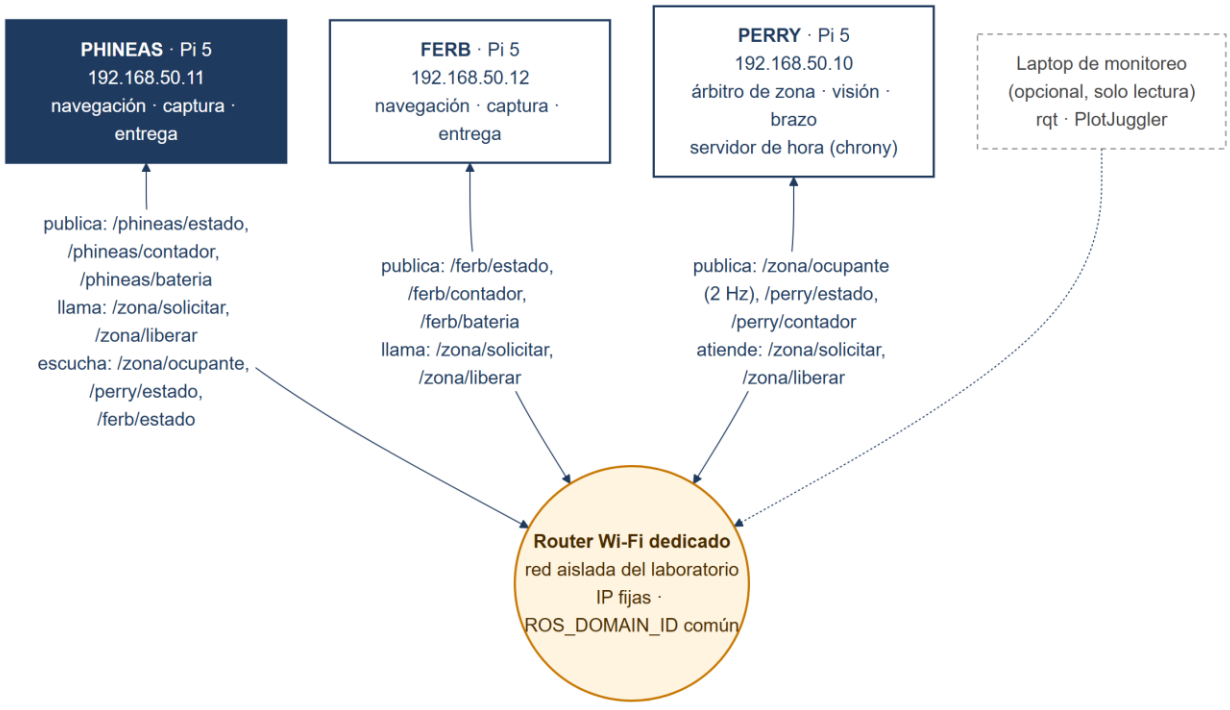
Comunicación con Ferb y Perry

Red y Descubrimiento

Las tres Pi se conectan a un router Wi-Fi dedicado con direcciones IP fijas y el mismo `ROS_DOMAIN_ID` (Figura 10). El descubrimiento de DDS usa multicast, que muchas redes universitarias bloquean, y el síntoma es que los robots se ven en casa y no en el laboratorio sin ningún error visible. Si el multicast falla también en el router propio, las tres Pi cambian a `rmw_zenoh_cpp`, que descubre a los pares mediante un router Zenoh (ros2, s. f.); el cambio debe hacerse en las tres a la vez, porque dos RMW distintos no se comunican. Perry actúa además como servidor de hora con chrony.

Figura 10

Topología de red y tópicos publicados por cada robot



Nota. Las direcciones IP son un ejemplo de plan de direccionamiento. Elaboración propia.

Contrato de Interfaces de Phineas

La Tabla 6 fija los mensajes que Phineas intercambia con el resto del sistema. La calidad de servicio, o QoS (Quality of Service), se elige según el uso: el ocupante del cuadro usa **transient local**, que entrega el último valor a quien se conecta tarde, para que un robot reiniciado sepa de inmediato quién tiene la llave.

Tabla 6

Interfaces ROS 2 de Phineas con el resto del sistema

Interfaz	Tipo	Rol de Phineas	QoS / uso
/phineas/estado	multirobot_interfaces/EstadoRobot	Publica (2 Hz)	Fiable, transient local
/phineas/contador	std_msgs/UInt32	Publica	Copia informativa
/phineas/bateria	sensor_msgs/BatteryState	Publica (1 Hz)	Mejor esfuerzo
/zona/solicitar	multirobot_interfaces/SolicitarZona (servicio)	Cliente	Concedida o denegada, con caducidad
/zona/liberar	multirobot_interfaces/LiberarZona (servicio)	Cliente	Incluye cubo_entregado y su color
/zona/ocupante	std_msgs/String	Suscriptor	Fiable, transient local; latido de 2 Hz
/ferb/estado, /perry/estado	multirobot_interfaces/EstadoRobot	Suscriptor	Prioridad y diagnóstico

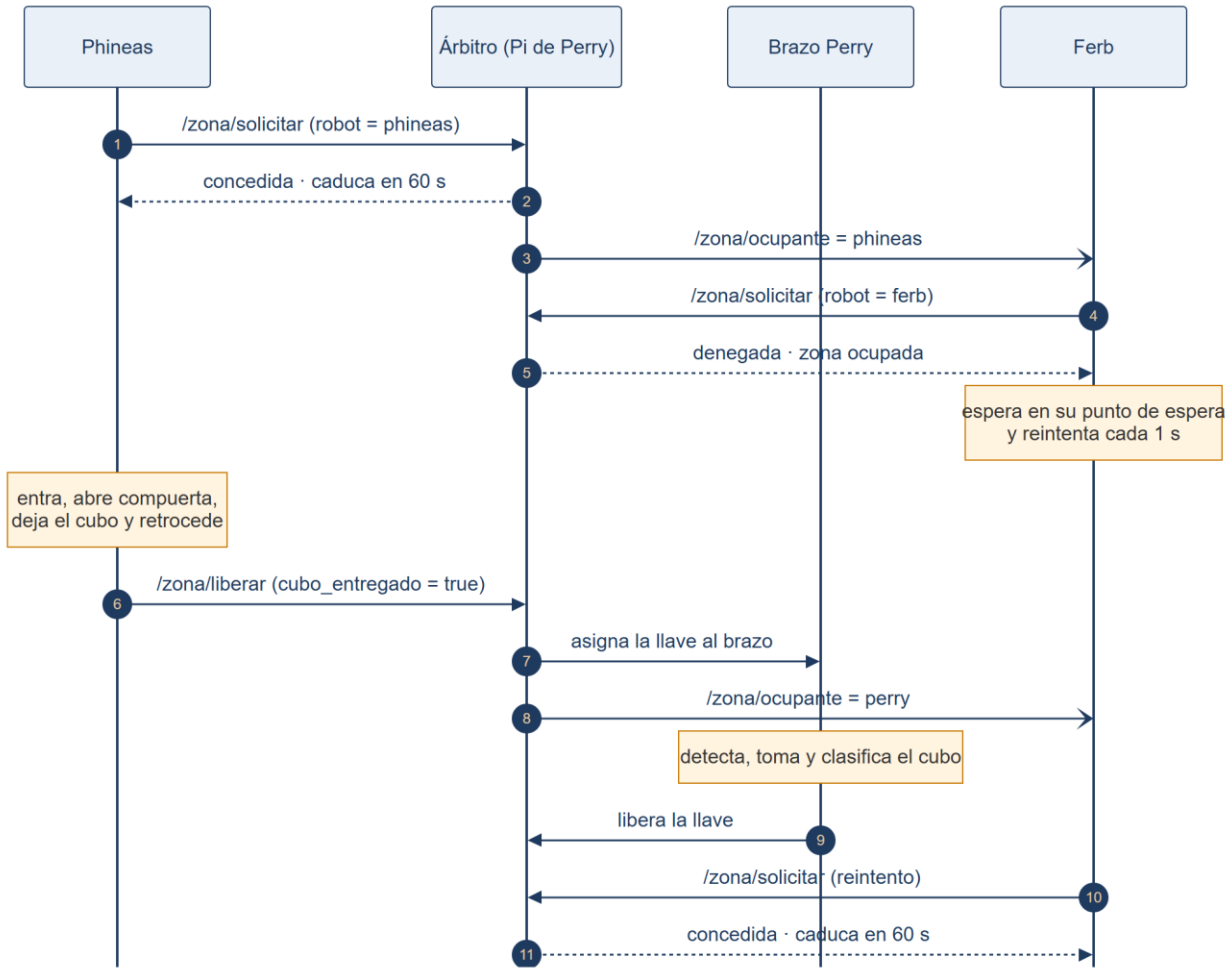
Nota. Nombres tomados del contrato del grupo (Ignacio, 2026a); el color en /zona/liberar es una ampliación propuesta.

Protocolo de Acceso a la Zona de Transferencia

El cuadro es un recurso de exclusión mutua: solo un robot o el brazo puede usarlo a la vez. Se gestiona con un **lease** (préstamo con caducidad) que concede un árbitro en la Pi de Perry, la única que está fija y enchufada. La Figura 11 muestra un caso con contención.

Figura 11

Secuencia de acceso al cuadro con contención entre Phineas y Ferb



Nota. Adaptado del mecanismo de llave del documento inicial del grupo (Ignacio, 2026a).

- La llave **caduca a los 60 s**: un robot averiado con la llave no paraliza el sistema.
- Phineas **libera la llave al salir del cuadro**, no al soltar el cubo; si la liberara antes, Ferb podría entrar mientras Phineas aún maniobra.
- El árbitro publica `/zona/ocupante` a 2 Hz y ese mensaje funciona como **heartbeat** (latido): sin él durante 2 s, Phineas pasa a Parada.

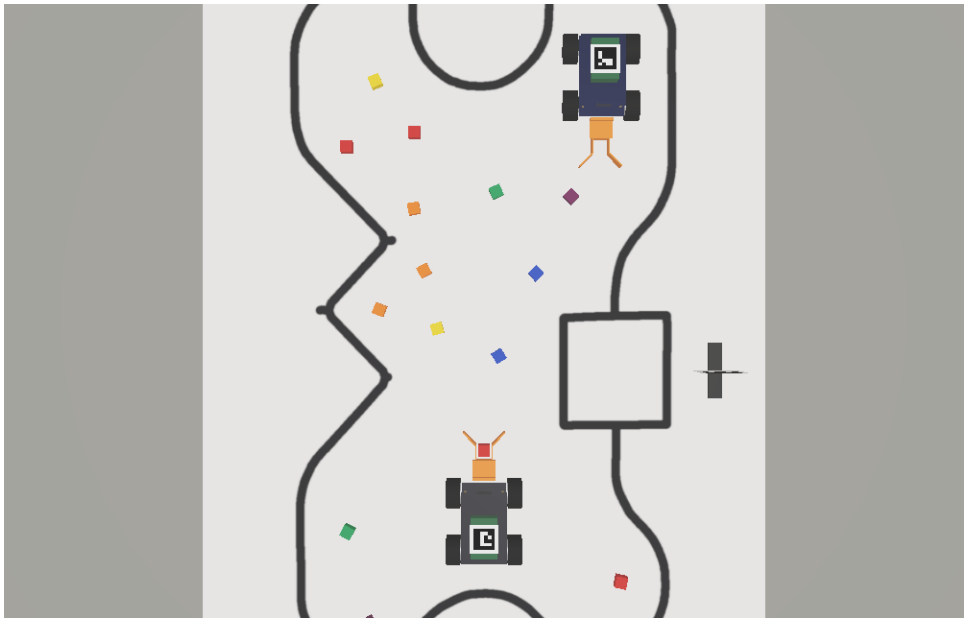
- El **conteo oficial de cubos** lo lleva el árbitro al recibir `/zona/liberar` con `cubo_entregado = true`. Una sola fuente de verdad evita contadores desalineados si un robot se reinicia.

Simulación en Webots

El grupo modeló la pista, los catorce cubos y los dos recolectores en Webots R2025a (Figura 12). El modelo reproduce el chasis de 186 × 161 mm con ruedas de 65 mm, los motores sin encoder, la pala en U con su servo y todos los sensores del robot real con los mismos nombres (Equipo Phineas, 2026). Las ruedas mecanum se simulan con fricción direccional a 45°, una técnica que también usa el modelo del KUKA youBot incluido en Webots, y patinan algo al girar, como las reales.

Figura 12

Vista cenital del mundo de Webots con Phineas y Ferb



Nota. Captura del mundo pista_ucab.wbt del grupo. Tomado de Equipo Phineas (2026).

La Tabla 7 relaciona cada dispositivo simulado con su pieza real. Mantener los mismos nombres hace que el código de los nodos no distinga entre simulación y realidad.

Tabla 7

Dispositivos del modelo de Webots y su equivalente real

Dispositivo en Webots	Pieza real	Qué devuelve
motor_fl, motor_fr, motor_rl, motor_rr	Motores TT por el HAT Cokoino	Se controlan por velocidad (rad/s)
servo_pala	SG90	Posición: 0 abajo, −0,7 rad arriba
camara	IMX219 de 120° (el modelo usa todavía 54°)	Imagen de 640 × 480
linea_fl, linea_fr, linea_rl, linea_rr	TCRT5000	Valor alto en blanco, bajo en la línea
ultrasonido	HC-SR04	Distancia de 0,02 a 4 m
ir_pala	FC-51	1 si hay cubo en la pala
color_pala	TCS34725	Imagen de 4 × 4 que se promedia a RGB
acelerometro, giroscopio	MPU-6050	m/s² y rad/s; el rumbo se integra y deriva

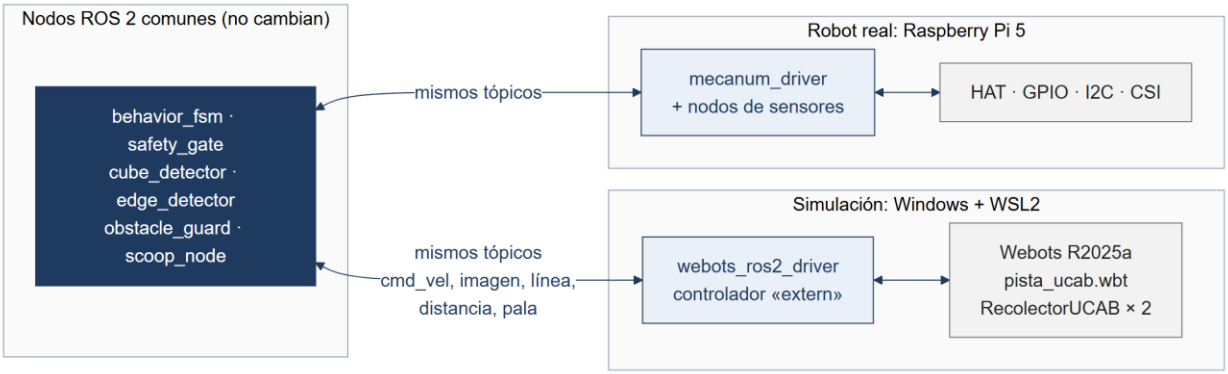
Nota. Datos del README del modelo (Equipo Phineas, 2026). El campo de visión de la cámara debe actualizarse al de la IMX219.

Del Simulador al Robot Real

El puente con ROS 2 es el paquete `webots_ros2`. En Windows, Webots se ejecuta en el sistema anfitrión y ROS 2 dentro de WSL2 con Ubuntu 24.04, y el paquete se instala con `ros-jazzy-webots-ros2` (Open Robotics, s. f.). El controlador del robot se cambia a externo y el driver de `webots_ros2` publica los dispositivos como tópicos. La Figura 13 muestra la consecuencia de diseño: los nodos de decisión y percepción son los mismos en ambos entornos, y solo se reemplaza la capa que habla con el hardware.

Figura 13

Mismos nodos de ROS 2 sobre Webots y sobre el robot real



Nota. Elaboración propia.

La simulación sirve para validar la lógica: la máquina de estados, las reglas de circulación, el protocolo de la llave y los encuentros con Ferb. No reproduce bien el patinaje real de las ruedas sobre el tablero, los colores con la luz del laboratorio, la caída de voltaje de la batería ni el calentamiento de los motores. Por eso cada prueba se aprueba primero en Webots y después se repite en la pista real. Existe además un problema conocido de conexión entre WSL2 y Webots en Windows (cyberbotics, s. f.), que conviene probar el primer día.

Riesgos y Verificaciones Pendientes

Tabla 8

Riesgos principales de Phineas

Riesgo	Mitigación	Verificación
El HAT no responde en la Pi 5	Portar el control a lgpio; si falla, pedir otro driver	Prueba de banco: un motor gira por software
La cámara CSI no funciona bien en Ubuntu 24.04	Plan B: Raspberry Pi OS con ROS 2 en Docker	Capturar una imagen desde Ubuntu el primer día
Choques con Ferb en el espacio compartido	Seis capas de convivencia; HC-SR04 trasero	Cruces de frente y en ángulo en Webots y en la pista
ArUco laterales no detectados por distancia o luz	Marcadores más grandes; el ultrasonido frena igual	Medir la distancia máxima de detección

Riesgo	Mitigación	Verificación
Confusión de colores por la luz del laboratorio	Calibración HSV en YAML; confirmación con TCS34725	20 cubos de los 6 colores: aciertos ≥ 95 %
El FC-51 no detecta cubos oscuros	Ajustar el trimmer; respaldo con el canal de brillo del TCS34725	Probar la captura con cubos de los 6 colores
La pala de Ferb confundida con un cubo naranja	Imprimir las palas en gris o negro	Imagen con Ferb en el encuadre
Reinicios de la Pi por caída de tensión	Rampas de aceleración; plan B: convertidor de 5 V y 5 A	Medir el pin 2 en un arranque brusco
Cubos reales más grandes que la U de 32 mm	Medirlos y ajustar la pala antes de imprimir	Medición con calibrador
Diferencias entre Webots y el robot real	Validar cada prueba en ambos entornos	Comparar tiempos y tasas de éxito

Plan de Pruebas

Cada requisito tiene una prueba observable que se ejecuta primero en Webots y después en la pista real. Los criterios provienen del documento de requisitos del grupo (Ignacio, 2026a), ajustados al reparto por colores y al espacio compartido.

Tabla 9

Pruebas de aceptación de Phineas

ID	Requisito	Criterio de aceptación
RF-01	No sale de la pista	5 min de movimiento continuo sin cruzar el borde
RF-02	Localiza cubos de sus colores	Encuentra 3 de 5 cubos propios en 3 min con cubos ajenos presentes
RF-03	Ignora los cubos ajenos	Ningún cubo verde, azul o morado llega al cuadro en 10 intentos
RF-04	Entrega el cubo en el cuadro	8 de 10 cubos quedan dentro del cuadro
RF-05	Pide turno antes de entrar	Con la llave denegada, espera en E_1 y no entra
RF-06	Se retira tras entregar	Sale por el sur antes de liberar la llave
RF-07	Esquiva a Ferb sin detenerse	10 cruces con Ferb sin contacto; ninguno queda detenido más de 3 s
RF-08	Se detiene sin enlace	Al apagar el router, se detiene en menos de 2 s

Conclusiones

Phineas puede construirse con el hardware disponible si se añaden los sensores de línea, el ultrasonido, la pala con su servo, el FC-51, el TCS34725, la IMU y la protección eléctrica mínima. Las dos correcciones más importantes frente al material del fabricante siguen vigentes: usar solo baterías 2S, por el límite de 10,8 V del DRV8833, y mover el PWM de nSLEEP a las entradas de los puentes.

Que la pista sea compartida desplaza la complejidad del proyecto hacia la convivencia entre robots. Los dos robots se mueven libremente y se esquivan por percepción local: cada uno ve al otro con su cámara gracias a marcadores ArUco laterales y se aparta hacia su derecha con un desplazamiento mecanum, sin detenerse. El frenado por ultrasonido y la prioridad fija cubren los casos que la cámara no resuelve, y una cámara cenital, si se aprueba, añadiría posiciones compartidas sin reemplazar lo anterior. Webots permite ensayar todo eso antes de arriesgar el hardware, siempre que se recuerde lo que no reproduce. El siguiente paso es la prueba de banco del HAT y de la cámara sobre la Pi 5 con Ubuntu.

Referencias

Cirkit Designer. (s. f.). *How to use sensor FC-51 IR: Pinouts, specs, and examples*. https://docs.cirkitdesigner.com/component/794e0aa1-9872-489b-bdc1-5f95d749086a/sensor-fc-51-ir

Cokoino. (s. f.-a). *CKK0018: Pi Power & 4WD HAT* [Repositorio de código, esquemático y ejemplos]. GitHub. https://github.com/Cokoino/CKK0018

Cokoino. (s. f.-b). *Introduction of Cokoino 4WD Robot Hat* [Manual técnico]. GitHub. https://github.com/Cokoino/CKK0018/tree/main/Tutorial

cyberbotics. (s. f.). *Webots ROS2 program running in Ubuntu (Windows WSL2) is not able to connect to Webots host simulator in Windows 11* (Issue #1031) [Reporte de error]. GitHub. https://github.com/cyberbotics/webots_ros2/issues/1031

Equipo Phineas. (2026). *Simulación de los recolectores en Webots* [Modelo de simulación y documentación inédita]. Robótica, Universidad Católica Andrés Bello.

Gadgetoid. (s. f.). *Hardware PWM on the Raspberry Pi 5* [Gist]. GitHub. https://gist.github.com/Gadgetoid/b92ad3db06ff8c264eef2abf0e09d569

GPIO Zero. (s. f.). *API - Pins*. https://gpiozero.readthedocs.io/en/latest/api_pins.html

Ignacio. (2026a). *Sistema multirrobot colaborativo: Documento inicial* [Documento inédito]. Robótica, Universidad Católica Andrés Bello.

Ignacio. (2026b). *Sistema multirrobot: Tecnología, código y to-do de laboratorio* [Documento inédito]. Robótica, Universidad Católica Andrés Bello.

joan2937. (s. f.). *lgpio: Python module documentation*. https://abyz.me.uk/lg/py_lgpio.html

LewanSoul. (s. f.). *LewanSoul mecanum robot chassis car kit with 360° omnidirectional wheel, aluminum alloy frame, TT motor* [Página de producto]. Amazon. https://www.amazon.com/dp/B093WDD9N5

Open Robotics. (s. f.). *Installation (Windows)* [Webots]. ROS 2 Documentation: Jazzy. https://docs.ros.org/en/jazzy/Tutorials/Advanced/Simulators/Webots/Installation-Windows.html

Open Robotics. (2024, 23 de mayo). *ROS 2 Jazzy Jalisco released*. https://www.openrobotics.org/blog/2024/5/ros-jazzy-jalisco-released

Raspberry Pi Ltd. (s. f.). *Raspberry Pi hardware*. Raspberry Pi Documentation. https://www.raspberrypi.com/documentation/computers/raspberry-pi.html

ros2. (s. f.). *rmw_zenoh: RMW for ROS 2 using Zenoh as the middleware* [Repositorio de código]. GitHub. https://github.com/ros2/rmw_zenoh

SparkFun Electronics. (s. f.). *HC-SR04 ultrasonic sensor* [Hoja de datos]. Digi-Key. https://www.digikey.com/htmldatasheets/production/1979760/0/0/1/hc-sr04.html

Taheri, H., Qiao, B., & Ghaeminezhad, N. (2015). Kinematic model of a four mecanum wheeled mobile robot. *International Journal of Computer Applications, 113*(3), 6–9. https://doi.org/10.5120/19804-1586

Texas Instruments. (2015). *DRV8833 dual H-bridge motor driver* (SLVSAR1E) [Hoja de datos]. https://www.ti.com/lit/ds/symlink/drv8833.pdf

Waveshare. (s. f.-a). *IMX219 camera module for Raspberry Pi 5, 8MP, MIPI-CSI interface, options for 79.3° / 120° FOV*. https://www.waveshare.com/pi5-imx219.htm

Waveshare. (s. f.-b). *TCS34725 color sensor* [Wiki]. https://www.waveshare.com/wiki/TCS34725_Color_Sensor